

5054 Homework 3

LAN Tianwei

tlanaa@connect.ust.hk

November 10, 2025

Problem 1: Basic Knowledge

1. Show explicitly that how fitting a cubic spline regression can be solved by linear regression with equality constraints.

A cubic spline regression can be written as

$$f(x) = \sum_{j=0}^3 \beta_j x^j + \sum_{j=1}^k \beta_{3+j} (x - \xi_j)_+^3$$

where $(x)_+ = \max(x, 0)$ is the truncated power basis. It's not difficult to find that $f(t)$ can be written in the form of vector multiplication:

$$\mathbf{y} = f(\mathbf{X}) = \mathbf{X}\boldsymbol{\beta}$$

where

$$\mathbf{X} = \begin{bmatrix} 0 & x_1 & x_1^2 & x_1^3 & (x_1 - \xi_1)_+^3 & (x_1 - \xi_2)_+^3 & \dots & (x_1 - \xi_K)_+^3 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & x_n & x_n^2 & x_n^3 & (x_n - \xi_1)_+^3 & (x_n - \xi_2)_+^3 & \dots & (x_n - \xi_K)_+^3 \end{bmatrix}$$

$$\boldsymbol{\beta} = (\beta_0 \ \beta_1 \ \beta_2 \ \beta_3 \dots \beta_{K+3})^\top$$

So the unconstrained least squares problem can be directly derived as:

$$\min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$$

Still, to ensure $\boldsymbol{\beta}$ corresponds to a **standard cubic spline**(i.e. the basis functions are linearly independent and have no redundant low-order terms), a linear equality constraint matrix $\mathbf{C}$ is imposed:

$$\mathbf{C}\boldsymbol{\beta} = \mathbf{0}$$

$$\mathbf{C} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 1 & \dots & 1 \\ 0 & 0 & 0 & 0 & \xi_1 & \xi_2 & \dots & \xi_n \\ 0 & 0 & 0 & 0 & \xi_1^2 & \xi_2^2 & \dots & \xi_n^2 \end{bmatrix}$$

which is to ensure that $\sum_{j=1}^K \beta_{3+j} (x - \xi_j)_+^3$ does not contain components lower than cubic, so as not to overlap with the 4 terms at the beginning. Therefore, cubic spline regression is equivalent to solving the following linear regression problem with equality constraints:

$$\min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2 \quad s.t. \ \mathbf{C}\boldsymbol{\beta} = \mathbf{0}$$

This is a quadratic programming problem, whose solution can be obtained by the Lagrange multiplier method:

$$\begin{bmatrix} \mathbf{X}^\top \mathbf{X} & \mathbf{C}^\top \\ \mathbf{C} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \boldsymbol{\beta} \\ \lambda \end{bmatrix} = \begin{bmatrix} \mathbf{X}^\top \mathbf{y} \\ \mathbf{0} \end{bmatrix}$$

2. Explain the difference between *piecewise polynomial regression* and *local polynomial regression*.

Piecewise polynomial regression divides the domain of the independent variable into several fixed intervals and fits an independent polynomial in each interval.

Local polynomial regression is to fit a polynomial using weighted least squares to a certain points on the domain. The fitting result is only useful for that point and is not a global function.

Table 1: Difference between Piecewise Regression and Local Polynomial Regression

Feature	Piecewise	Local
Model Form	Global piecewise definition	Local dynamic fitting
Smoothness	Often discontinuous	Naturally smooth
Number of Parameters	Finite (fixed)	Infinite (non-parametric)
Key Parameter	Knots	Bandwidth
Computational Efficiency	High (single fit)	Low (refit per point)

3. Does increasing the bandwidth h in local polynomial regression increase or decrease the bias? How about the variance? Explain with details.

When bandwidth h increases, the regression estimate $\hat{f}(x_0)$ at any point x_0 is computed using a larger neighborhood of data points. This lead to **lower variance** because averaging over more observations reduces the sensitivity to random noise in the training data. However, it also introduces **higher bias**, because the local polynomial (e.g., constant or linear) may fail to capture the true curvature of $f(x_0)$ over a wide interval, resulting in a systematic deviation of $\mathbb{E}[\hat{f}(x_0)]$ from the true value $f(x_0)$.

When h decreases, $\hat{f}(x_0)$ relies on fewer nearby points, making the fir more sensitive to noise and **making the variance increase**, while it also allows the model to better capture local features of $f(x)$, **reducing bias**.

4. Regression tree is most similar to which of the following methods: (a) KNN regression (b) Piecewise constant regression (c) Logistic regression, and explain the difference between regression tree and the method you choose.

A regression tree is essentially a form of **(b) piecewise constant regression**. It partitions the input space into disjoint, axis-aligned regions (defined by the tree's splits), and within each region—corresponding to a leaf node—it predicts a constant value, typically the mean of the target

variable for training samples in that region. This results in a globally defined function that is constant over each subregion, which precisely is the definition of a piecewise constant model. The tree structure simply provides a data-driven, hierarchical way to construct those regions.

In contrast, (a) KNN regression is a *lazy, local averaging method* that computes predictions on the fly using the nearest neighbors of a query point; it does not produce a fixed partition or explicit function. (c) Logistic regression, on the other hand, is a parametric, linear model for classification with a completely different objective and functional form. Thus, neither KNN nor logistic regression shares the core piecewise-constant, globally partitioned nature of regression trees.

5. When a tree gets finer, how does the bias and variance behave? Explain with details.

When the regression tree gets finer, it's like breaking up the piecewise constant regression into more parts. This makes the model make predictions based on fewer nearby points and is more sensitive to noise and outliers, resulting in increased variance and decreased bias, which is prone to overfitting.

6. Generally speaking, which of the following methods can have the smallest bias? (a) Linear regression (b) Regression tree (c) Natural splines, and explain with details.

If the true function is *smooth* and *non-linear*, (c) **Natural Splines** typically HAVE THE LOWEST BIAS because they can fit nonlinearity while maintaining smoothness and continuity.

However, if the true relationship is *inherently linear*, (a) **Linear Regression** may actually has the lowest bias.

Because (c) **Regression Tree** is piecewise constant, even increasing the tree depth makes it difficult to approximate smooth curves. Therefore, in smooth scenarios, the bias is usually higher than that of natural splines.

7. Write down the formal and explicit definition of variable importance measure for random forest using math notations. How can we do variable selection/model selection in random forest?

In the random forest variable importance metric, the commonly used indicator is **Permutation Importance**. All mathematical symbols used are as Table 2.

The permutation importance of variable j is defined as

$$\mathbf{VI}_j^{(\text{OOB})} = \frac{1}{B} \sum_{b=1}^B \left[\frac{1}{\mathcal{O}_b} \sum_{i \in \mathcal{O}_b} \{L(y_i, f_b(\mathbf{x}_i)) - L(y_i, f_b(\tilde{\mathbf{x}}_i^{(j)}))\} \right]$$

where

$$\tilde{\mathbf{x}}_i^{(j)} = (x_{i1}, \dots, x_{i,j-1}, \pi_j(x_{ij}), x_{i,j+1}, \dots, x_{ip})$$

which means that only the value of the j -th variable in the OOB sample is randomly replaced, and the other variables remain unchanged. In other words, PI measures the average increase in prediction error after destroying the original relationship between X_j and Y .

Table 2: Notation Table for Random Forest Variable Importance

Symbol	Description
$\mathcal{D}_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$	Training sample, $\mathbf{x}_i \in \mathbb{R}^p$
b	Tree index, $b = 1, \dots, B$
$\mathcal{S}_b$	Bootstrap sample for tree b
$\mathcal{O}_b = \{i : i \notin \mathcal{S}_b\}$	Out-of-bag (OOB) indices for tree b
$f_b(\mathbf{x})$	Prediction of tree b at $\mathbf{x}$
$L(y, f)$	Loss function (e.g. squared or 0–1)
j	Variable index, $j = 1, \dots, p$
$\pi_j(\cdot)$	Permutation operator on j -th coordinate
$\tilde{\mathbf{x}}_i^{(j)}$	$\mathbf{x}_i$ with x_{ij} permuted via π_j
$\text{VI}_j^{(\text{OOB})}$	OOB permutation importance of variable j

Next, introduce 2 variable-importance-based variable selection strategies in random forest:

One is **Single-threshold**. Choose significance level α (e.g. 0.05), then compute the $(1 - \alpha)$ quantile $q_{1-\alpha}$ of the null-importance distribution by Monte-Carlo. Retain $\mathcal{M} = \{j : \text{VI}_j \geq q_{1-\alpha}\}$.

The other one is **Stepwise Backward Elimination** as the following steps:

- 1. Initial model $\mathcal{M}_0 = \{1, \dots, p\}$.
- 2. At step k :
 - compute $\{\text{VI}_j^{(k)}\}_{j \in \mathcal{M}_k}$;
 - drop the least important: $j^* = \arg \min_{j \in \mathcal{M}_k} \text{VI}_j^{(k)}$;
 - update $\mathcal{M}_{k+1} = \mathcal{M}_k \setminus \{j^*\}$.
- 3. Stop when a preset number of variables is reached or cross-validated error increases significantly.

Problem 2: Investigation of the Diameter, Height and Volume for Black Cherry Trees

1. Fit four polynomial models (deg=1,2,3,4) to predict the Volume using Girth. Choose the model with the largest adjust R-squared. Plot the polynomial function of the model and also plot the confidence bands with ± 2 standard error. How about choosing the model using 5-CV error?

According to the code's output, the model with the maximum adjusted R-squared is at the degree of 2. The polynomial function equation is as follow:

$$\text{Volume} \approx 10.7863 - 2.0921 \cdot \text{Girth} + 0.2545 \cdot \text{Girth}^2$$

And the plot with confidence bands is shown in Figure 1.

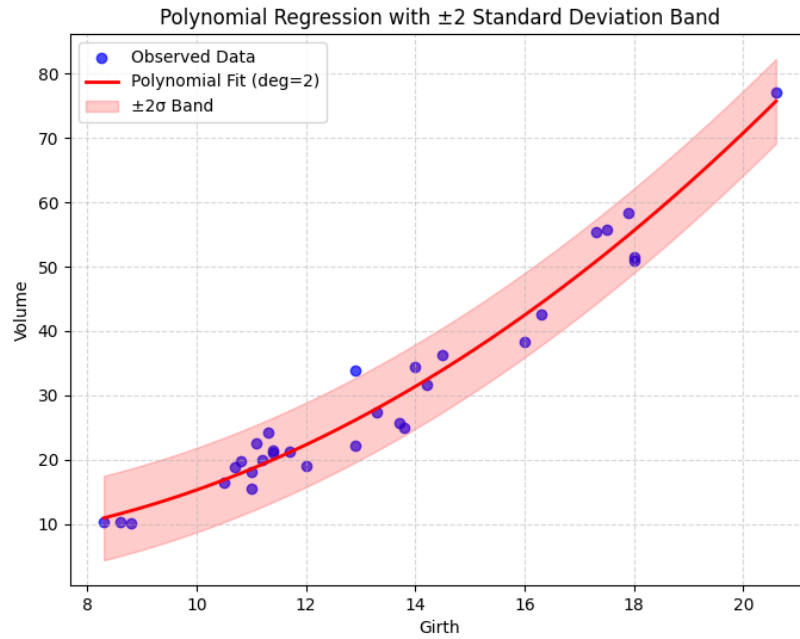


Figure 1: Polynomial model at degree of 2

On the other hand, when 5-fold cross-validation is used to find the optimal model, the indicator used is CV-R-squared. This indicator would not expand as the number of variables increases like the ordinary R-squared, so it can also be used to compare the performance of polynomial regression.

According to the code's output, the model with the maximum adjusted CV-R-squared is at the degree of **3**. The polynomial function equation is as follow:

$$\text{Volume} \approx -23.5184 + 5.7801 \cdot \text{Girth} - 0.3256 \cdot \text{Girth}^2 + 0.0137 \cdot \text{Girth}^3$$

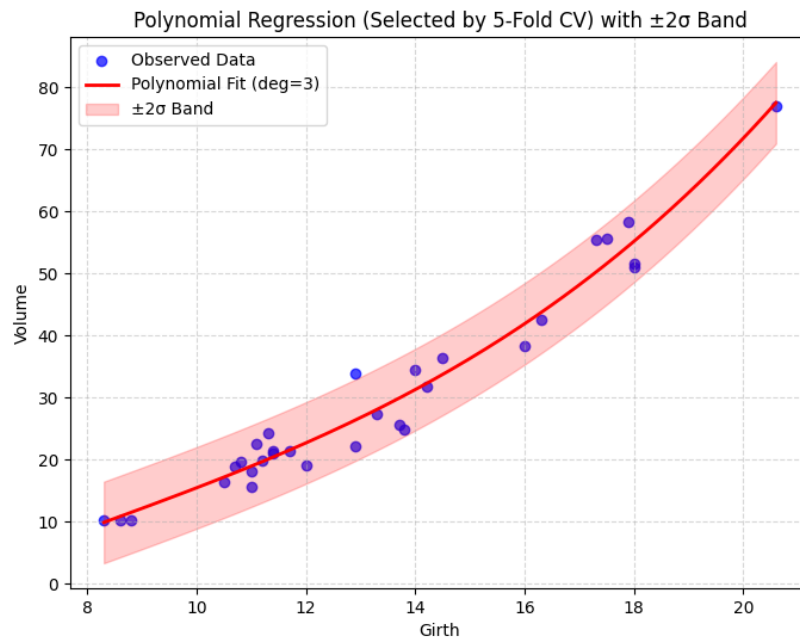


Figure 2: Polynomial model at degree of 3

2. Use a polynomial logistic regression model with $\text{deg}=2$ to predict whether the Volume is larger or not than 30, using the variable Girth. Plot the function $P(\text{Volume} > 30)$ with respect to Girth and the confidence bands with ± 2 standard error.

A polynomial logistic regression (degree = 2) was used to fit the tree data. The ± 2 standard error bands exhibit a trend of being *wide in the middle* and *narrow on both sides*, which is consistent with the typical shape of logistic regression confidence bands. Confidence bands outside the range (0, 1) have been clipped. The log-odds expression is as follows:

$$\log\left(\frac{P}{1-P}\right) = 5.6239 - 3.3205 \cdot \text{Girth} + 0.2121 \cdot \text{Girth}^2$$

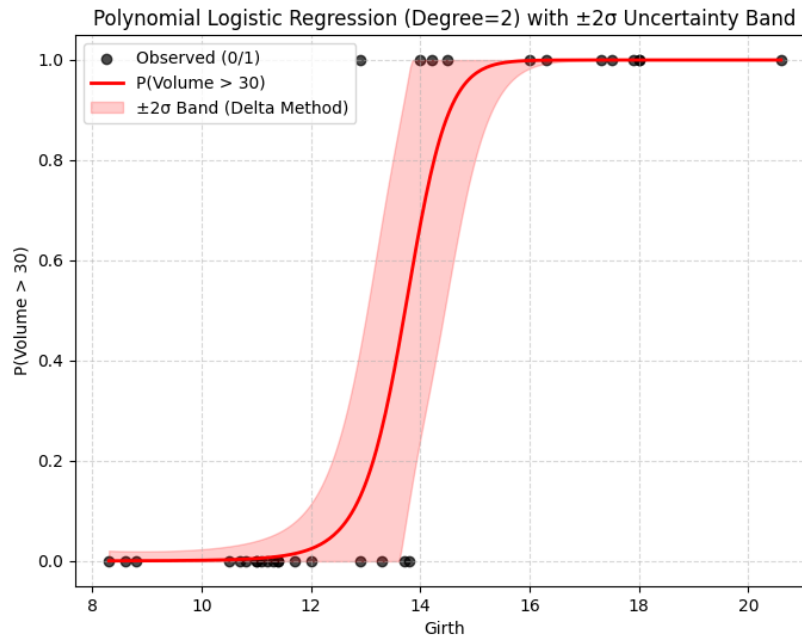


Figure 3: Polynomial Logistic Regression at degree of 2

3. Fit a regression spline with $\text{deg}=2$ to predict the Volume using the variable Girth at knots 101418. Plot the function and also the confidence bands with 2 standard error.

Using regression spline (degree = 2) to fit the data, it can be found that the more nodes there are, the more flexible the model is and the more nonlinear features it can capture, but the risk of overfitting also increases.

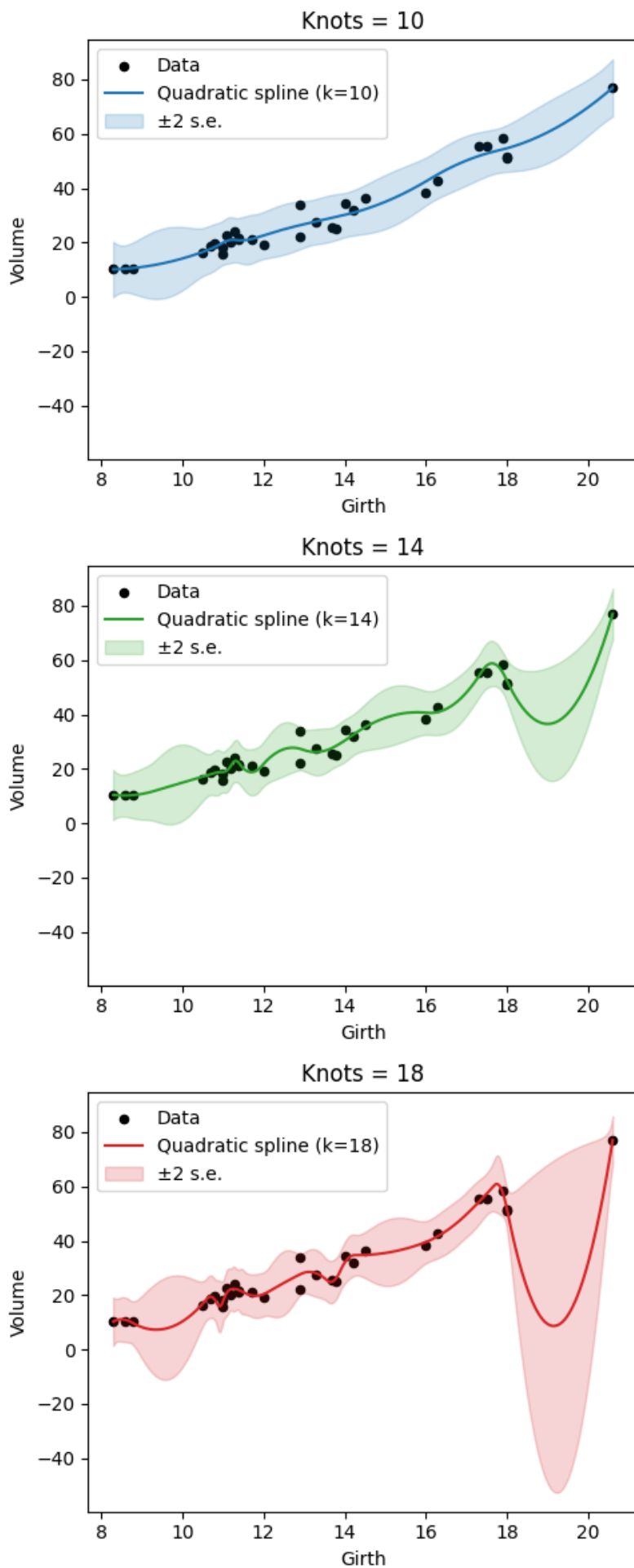


Figure 4: Regression Spline

4. Fit a smoothing spline to predict the Volume using the variable Girth where the smoothing level is chosen by Cross-Validation. Plot the function. What is the used degrees of freedom?

The smoothing spline fitting results are shown in Figure 6. The best model obtained by cross-validation has a λ of **23.36** and a degree of freedom of **4.10**.

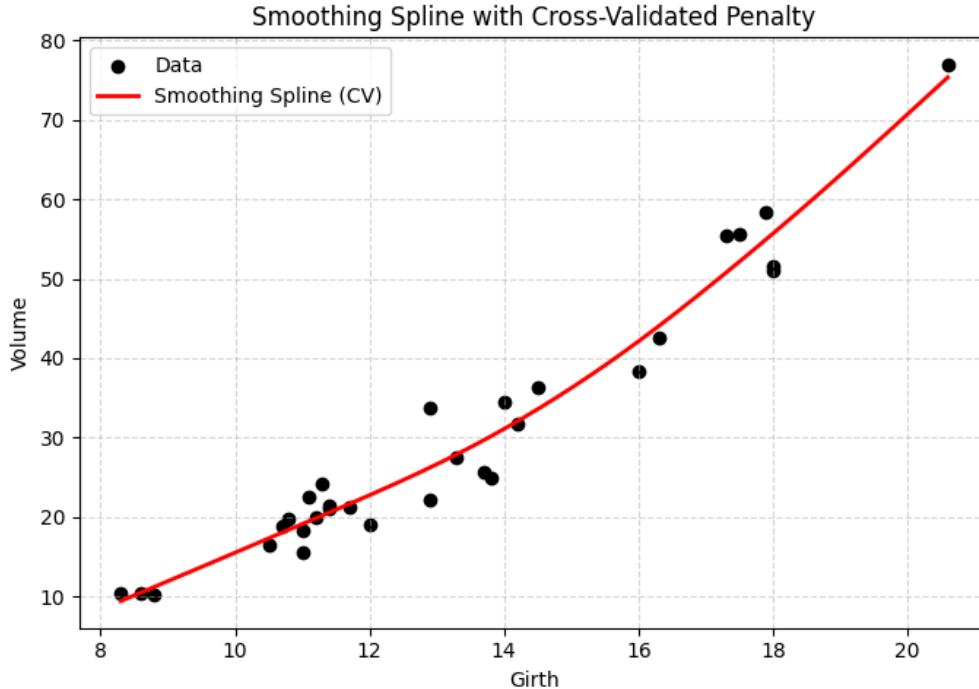


Figure 5: Best Smoothing Spline

5. Use both the variable Girth and Height to predict the Volume by a GAM where the individual function on Girth is a smoothing spline with $df=4$ and the function on Height is a smoothing spline with $df=5$. Plot the functions and also the confidence bands.

In GAM model, the closed-form function is

$$y = \beta_0 + s_1(\text{Girth}) + s_2(\text{Height})$$

which is a binary function of Girth and Height. After calculating the smooth functions of the two predictors, the results are shown in Figure 6. The two figures show their respective contributions, and they are not correlate to each other, so there is still a certain distance from the original data points.

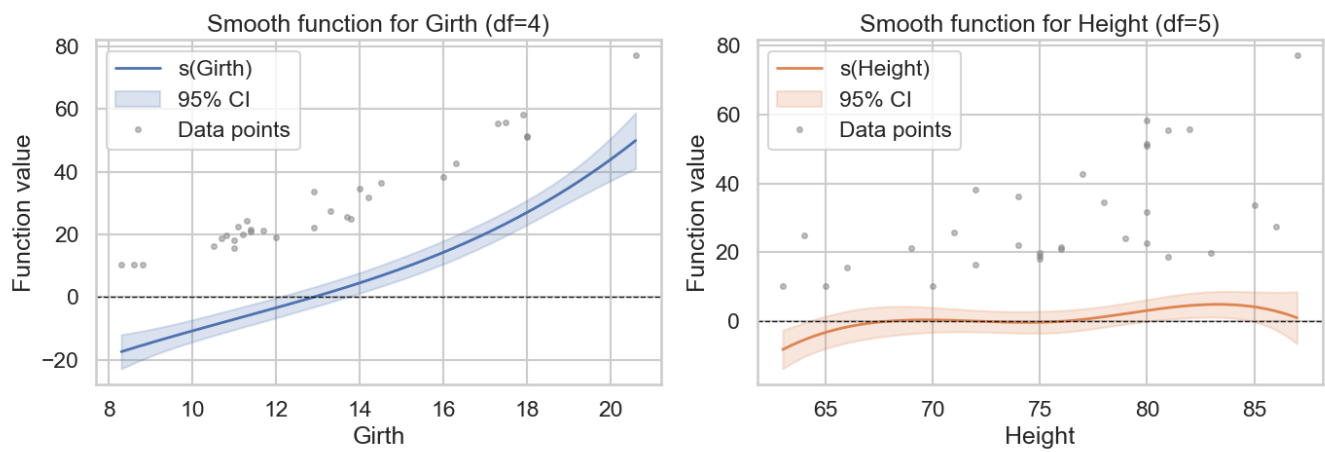


Figure 6: GAM respective contributions

Therefore, drawing two predictors and one responder into the same 3D graph can better show the overall performance of the entire model, as is shown in Figure 7.

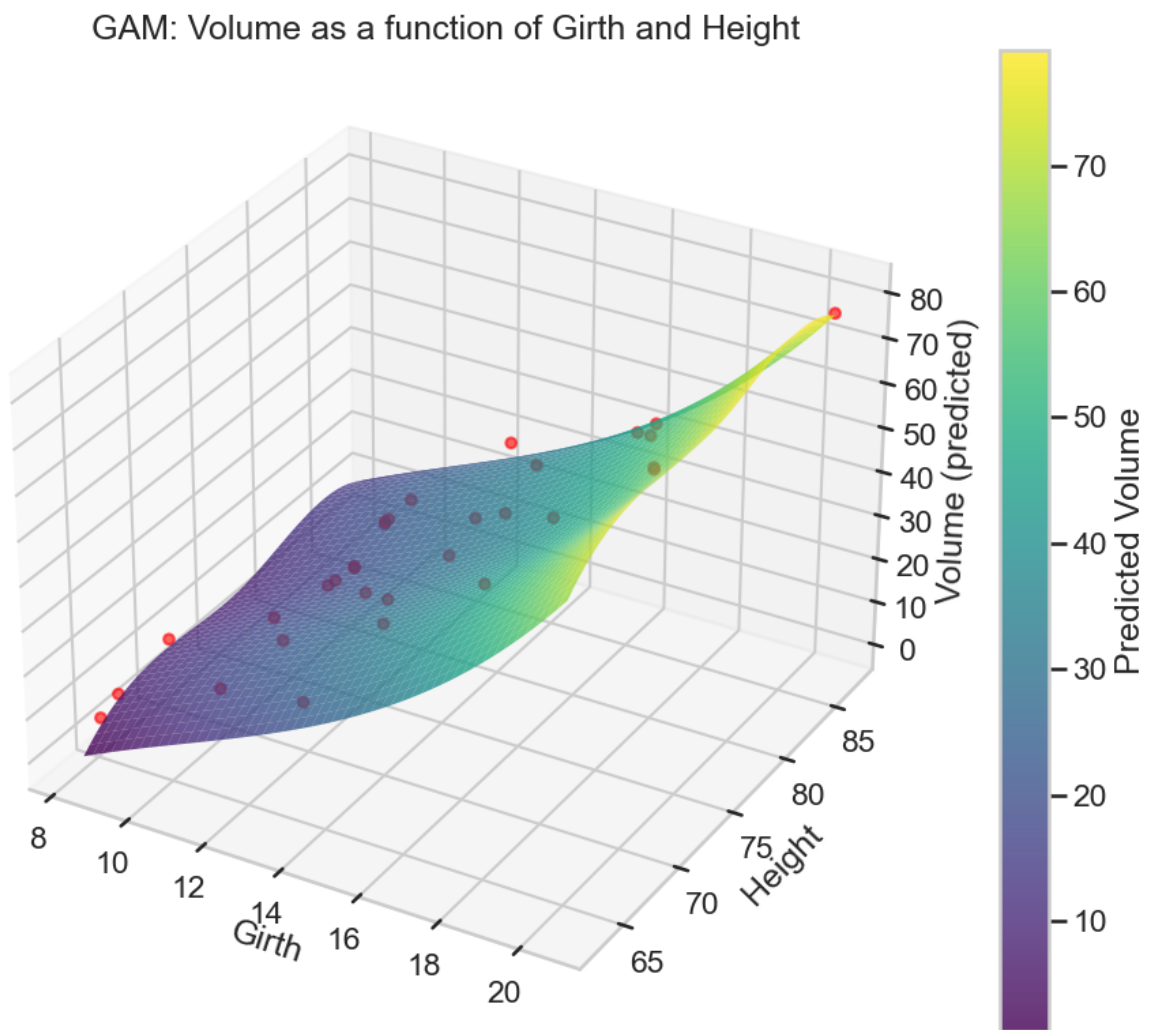


Figure 7: 3D Plot of GAM

Problem 3: Audit Risk

1. Use the train dataset to fit a classification tree and, plot the tree and report the training error. Test the performance on the test dataset and report the confusion matrix.

In the dataset, `LOCATION_ID` column is a high-cardinality discrete categorical variable. One-hot encoding on this column would lead to an overly sparse data matrix. Therefore, it was directly ignored in feature engineering. The training error obtained after fitting the model is 0.035. The classification tree diagram and confusion matrix are shown in Figures 8 and 9.

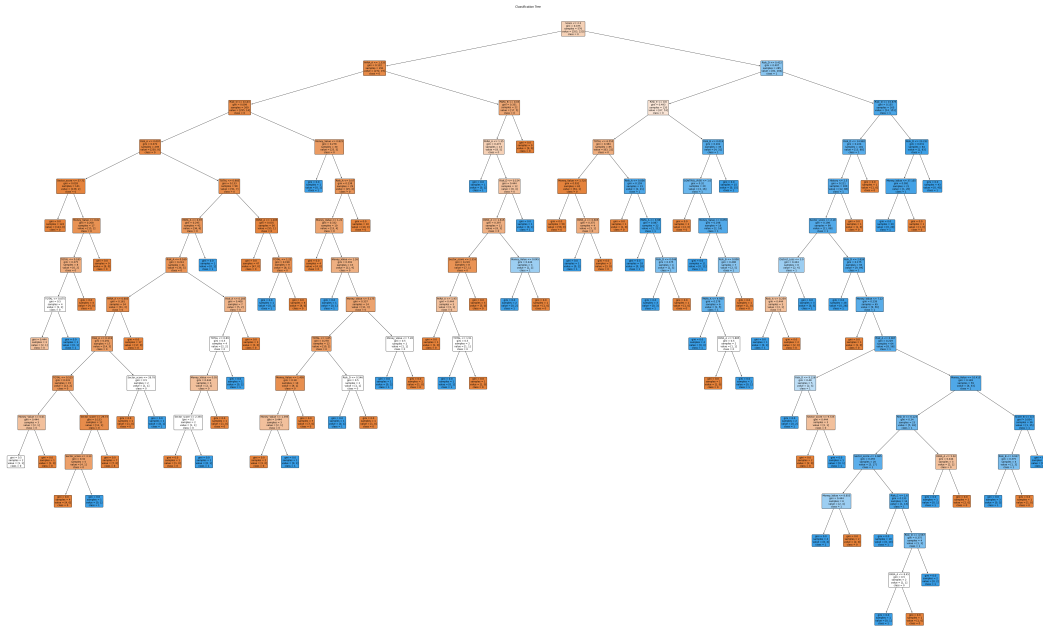


Figure 8: Classification Tree

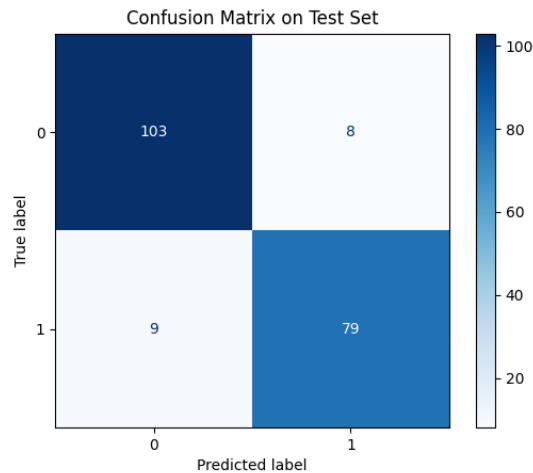


Figure 9: Confusion Matrix

2. Use CV to prune the tree in Step 1 on the train dataset. Plot the train error versus the tree size. Plot the pruned tree which has the best train error. Report the test error.

Cost-Complexity Pruning (CCP) is used in this section. As can be seen from Figure 10, the training error decreases sharply as the number of nodes increases. This is because each split of the classification tree only further purifies the training samples, and the training error cannot increase. In extreme cases, the tree becomes so large that each leaf contains only one sample, thus making the training error = 0.

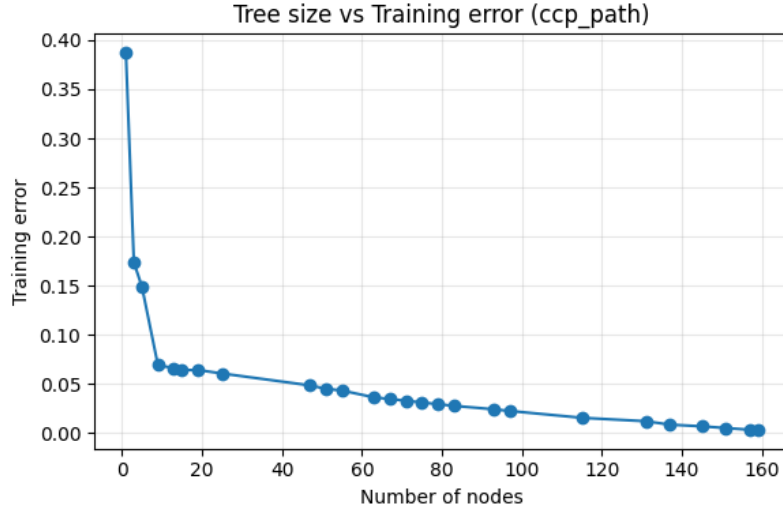


Figure 10: Relationship between tree size and training error

The optimal pruned model parameters selected based on the 5-fold cross-validation error are

$$\text{CCP-}\alpha = 0.003064, \text{err}_{\text{cv}} = 0.1249, \text{err}_{\text{test}} = 0.0503$$

and the tree is shown in Figure 10.

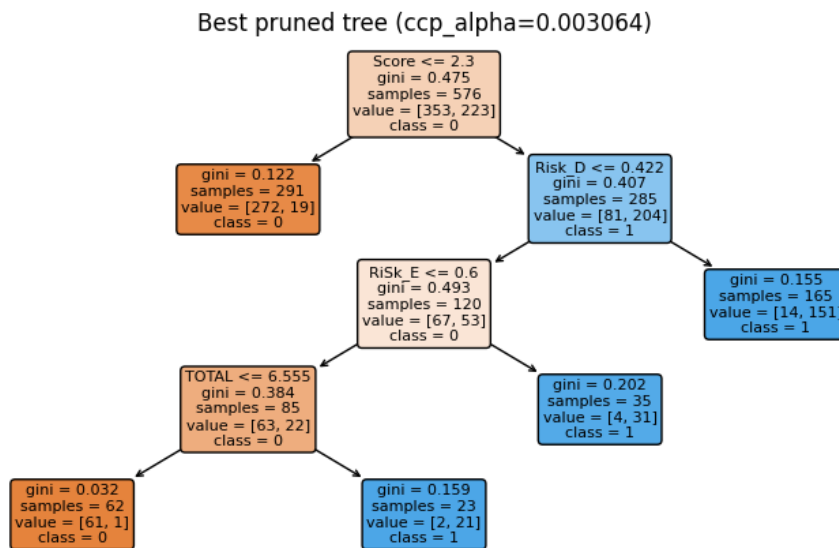


Figure 11: Best Pruned Tree

3. Use random forest on the train dataset to build a classifier to predict the risk where setting $m=13$ and $ntree=25$. Report the training error.

$$err_{tr} = 0.0052$$

4. Repeat Step 3 with four difference choices $m = 8, 12, 14, 16, 18$ and choose the one with smallest mis-classification error on the train dataset. Test its performance on the test dataset.

m	err_{tr}
8	0.0052
12	0.0069
14	0.0052
16	0.0069
18	0.0052
m_{best}	8 (min err_{tr} 0.0052)
err_{tr}	0.0503

Table 3: Training error and best random forest under different m

5. Compare the above methods and report any findings you observe.

Table 4 summarises the training and test errors of the three modeling strategies. The single classification tree gives the largest training error (3.5%) and a test error of 8.54%. After cost-complexity pruning, the training error drops to 0.5% and the test error improves markedly to 5.07%. The random forest with $m = 8$ and $ntree=25$ achieves almost the smallest training error (0.52%) and the best generalisation performance, with a test error of 5.03%.

Table 4: Training and test errors of the three approaches		
Model	Training error	Test error
Full classification tree	3.50%	8.54%
Best pruned tree (CCP- α)	0.50%	5.07%
Random forest ($m = 8$, $ntree=25$)	0.52%	5.03%

Although the absolute gap between the pruned tree and the random forest is small, the ensemble consistently stabilizes the prediction and is therefore recommended for this audit-risk task.

Problem 4: Implementing Regression Tree

On the first step, I used AI to generate basic regression tree code and a series of tool functions, including `MSE calculation` and `best splitting` function. I also designed a main function to load the code and fit the model. The calculated model had a test-error of 0.4918.

Then, I modified the original regression tree code, mainly including replacing MSE with MAE, changing the splitting criterion from mean to median, adding v_{gain} and leaf number parameters in class initialization, updating leaf nodes in the recursion, and adding a v_{gain} threshold stopping condition. Afterwards, I modified the main function to formatted output, tried different v_{gain} and maximum leaf number as inputs, and obtained the following test-error as output:

Table 5: Test MSE under different `vgain` and `max_leaves`

<code>vgain</code>	<code>max_leaves</code>				
	2	4	8	16	32
0.5	0.4431	0.4258	0.3140	0.3103	0.2510
1.0	0.4431	0.4258	0.3140	0.5187	0.5195
1.5	0.4431	0.4258	0.2982	0.5171	0.5192
2.0	0.4431	0.4258	0.2471	0.5421	0.5421
Best	vgain=2.0, max_leaves=8, Test MSE=0.2471				

The experiment shows that under the absolute-loss regression tree, both increasing v_{gain} and `max_leaves` exhibit a clear elbow: the test MSE reaches its minimum of 0.2471 at `max_leaves=8` and $v_{gain} = 2.0$, while further enlarging the tree or reducing `vgain` raises the error again because of over-fitting. Thus, moderate pruning ($v_{gain} \approx 2.0$) together with a medium model size (8 leaves) offers the best bias-variance trade-off and the optimal generalization performance.